Discover Electronics

RESEARCH Open Access

Fast and accurate identification of hardware Trojan locations in gate-level netlist using nearest neighbour approach integrated with machine learning technique

Anindita Chattopadhyay[1†], Siddharth Bisariya[2†] and Vijay Kumar Sutrakar[2*]

†Anindita Chattopadhyay and Siddharth Bisariya have equally contributed.

*Correspondence:
Vijay Kumar Sutrakar
vks.ade@gov.in
[1]Department of Electronics and Communication, BMS College of Engineering, Bengaluru, Karnataka, India
[2]Aeronautical Development Establishment, Defence Research and Development Organisation, Bengaluru, Karnataka, India

Abstract

Ensuring the security of integrated circuits (ICs) requires reliable detection and precise localization of Hardware Trojans (HTs), which remain challenging due to increasing design complexity and lack of golden references. This paper introduces a machine learning framework that integrates graph-based modeling, Graph Neural Networks (GNNs), and nearest neighbour (NN) enhancement for fast and accurate HT detection at the gate-level netlist without relying on the golden reference. Three different machine learning models are employed in the present work. Case-I uses a decision tree classifier with Principal Component Analysis (PCA) for binary detection of Trojan presence, as a reference model. The decision tree based machine learning model is initially validated against the formal verification method. The decision tree model is only able to identify HT presence without localizing the locations of HT in the circuit. Case-II uses a GNN-based graph-to-graph classification, distinguishing clean netlists from the infected one at sub-graph level (coarse grained). The model is able to map the infected sub-graphs back to the initial netlist circuit for the pinpointing of HTs at sub-graph level. Case-III uses a further simplified model with GNN-based node classification, enabling fine-grained localization of compromised gates in the circuit using only nodes. This model is ideal for pinpointing the exact Trojan locations within large-scale circuits. Subsequently, NN based concept is embedded with GNN models for further enhancing the detection accuracy of Case-II (accuracy improved from 62.8% to 97.7%) and Case-III (accuracy improved from 79.8% to 97.7%). Also, the scalability of the proposed approaches across diverse Trojan types, including combinational, sequential, and state-triggered attacks are validated by experiments carried on Trust-Hub benchmarks and Yosys-generated datasets. Comparative evaluation with state-of-the-art methods demonstrates superior performance, achieving 98.5% precision, 99.1% recall, and 96.7% F1-score, while maintaining computational efficiency. By combining graph structural learning with NN-based contextual refinement, the proposed work delivers a high-performance, architecture-agnostic solution for detecting and localizing hardware Trojans in modern ICs as tested against unknown designs.

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Keywords Graph-level netlist, Hardware trojan, Machine learning, Nearest neighbour, Path retrace algorithm

1 Introduction

Globalization has brought significant transformations in the entire supply chain of integrated circuits (ICs). Trojan insertion in ICs can occur either at (a) design stage, (b) synthesis stage, (c) fabrication stage, and (d) assembly and packaging stage. In the early design stage, untrusted third-party intellectual property (IP) cores or malicious engineers may embed additional logic that compromises system security. During synthesis, optimization, and design-for-test (DFT) insertion, gate-level netlists may be modified to introduce hidden trigger conditions or payload circuits. At the fabrication stage, untrusted foundries present one of the highest risks, as adversaries may manipulate mask layouts or transistor doping profiles, enabling stealthy parametric Trojans that subtly alter delay or power consumption. In the assembly and packaging stage, additional logic or rerouting can be inserted at the interconnect level, while the post-silicon testing and validation phase may also be exploited if compromised test facilities mask Trojan activity during functional verification. Moreover, the heavy reliance on overseas foundries for IC fabrication, coupled with the growing awareness of the vulnerability of chips to HT insertion in these foundries, has intensified security concerns [1]. In the recent past, several methods have been proposed for HT detections for ensuring the trustworthiness of the fabricated chips [2–4]. However, the primary obstacle encountered in previously developed non-destructive approaches is the accuracy of detection.

Evidence from past incidents shows that hardware compromises in the supply chain present practical, not just theoretical, security concerns. In 2007, during an Israeli airstrike on a suspected nuclear facility in Syria, the country's radar defense system reportedly failed to detect incoming aircraft. Several accounts suggested that the malfunction was not coincidental but rather the result of a remote kill switch embedded in the commercial off-the-shelf microprocessors powering the radar. While definitive technical evidence was never disclosed, the incident is widely cited as one of the earliest demonstrations of how hardware backdoor could be exploited in real-world military operations [5]. Another case was uncovered in 2012, when researchers from the University of Cambridge discovered an undocumented backdoor in the Actel/Microsemi ProASIC3 family of FPGAs. These devices were widely deployed in military, aerospace, and critical infrastructure applications. The backdoor enabled an adversary to extract encryption keys, reprogram the device, and bypass security features, thereby granting full control over the chip. This revelation confirmed long-standing fears that hardware Trojans could be intentionally or unintentionally embedded within commercial semiconductor products, presenting severe risks to defense-grade electronics [6]. In 2015, Juniper Networks disclosed that its firewall operating system, ScreenOS, contained two serious backdoors: a hardcoded master password and a modified random number generator (Dual_EC_DRBG) suspected of being compromised. While primarily a firmware vulnerability, its implementation was closely tied to the hardware cryptographic accelerators in Juniper's devices, making it partly a hardware-linked threat. These backdoor allowed attackers to decrypt VPN traffic and gain unauthorized access to network devices. The Juniper case

highlighted how hardware-firmware interfaces can be manipulated to introduce stealthy and persistent security weaknesses [7].

Consequently, to tackle the challenge of detecting HTs, it becomes necessary to perform chip-level reverse engineering. However, this process is very expensive. Nowadays, advanced imaging-based techniques are a promising option for driving the complete netlist from a fabricated chip [8]. This also helps the designers for comparing the changes with their original design and identifying any changes, if any, incorporated during the chip fabrication process. However, such techniques are not applicable to Commercial Off-The-Shelf (COTS) ICs and System- on-Chips (SoCs) incorporating third-party Intellectual Properties (IPs), due to the absence of golden models. COTS ICs and SoCs require more sophisticated design analysis tools. Recent developments in this area have seen the emergence of various tools [9]. However, these tools consider all the internal registers uniformly and attempt to recognize functional logic through pattern matching, resulting in a mixture of control logic and data path within the identified functional components. This complicates the task for testers in fully recovering the design's functionality. Hence, identification and pin- pointing the exact locations of functional Trojans inside ICs are still a major area of concern.

In this paper, an attempt has been made to identify functional Trojans (especially their exact locations inside the circuits) that are inserted maliciously. The proposed approaches demonstrate a comprehensive methodology for detecting HTs in digital circuits, leveraging graph-based representation and machine learning techniques, as shown in Fig. 1. The process begins with a gate-level netlist, which contains the digital circuit in terms of logic gates and their interconnections. This netlist is then flattened, simplifying the design into a single level of logic gates and connections. The flattened netlist is subsequently transformed into a graph representation. This graph representation facilitates the application of GNNs and other machine learning models to detect anomalies indicative of HTs. The output from the proposed model identifies nodes and subgraphs that are potentially infected with HTs. Finally, the process involves locating the exact positions of these infected nodes within the original gate-level netlist, enabling precise identification of compromised parts of the circuit and facilitating subsequent corrective actions. This integrated approach combines traditional circuit design with advanced machine learning, leading to (a) enhancing the detection and mitigation of HTs and (b) ensuring the safety and truthfulness of digital systems. Further attempts have also

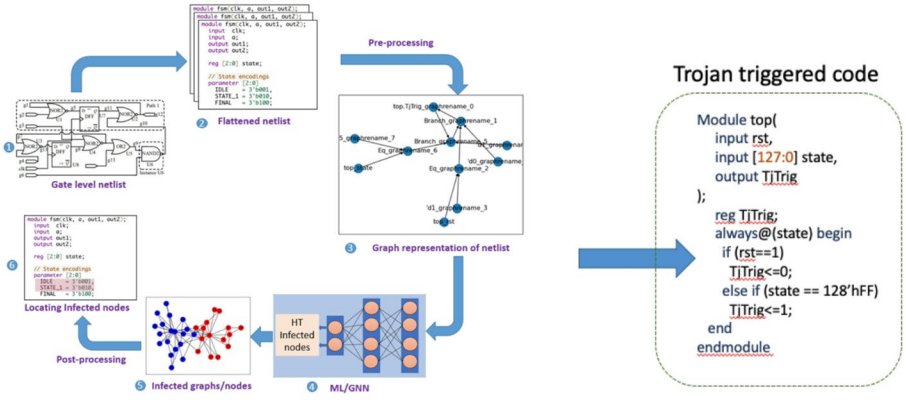


Fig. 1 ML method employed for HT detection and identification of its location inside the netlist

been made for improving the accuracy of the machine learning models utilizing nearest neighbour (NN) approach.

The subsequent sections of the paper are organized as follows: Section II provides an overview of current work related to state register identification and netlist classification. Section III offers a brief overview of the proposed methodology. In section IV, data sets and the details of the proposed model are presented. Results are discussed in the section V. Finally, Section VI concludes the paper with a summary and suggests potential applications of the proposed methodology.

2 Related work

The design of digital circuits commonly occurs at the Register Transfer Level (RTL). RTL provides the details of signal flow among various registers. RTL descriptions are then transformed into gate-level netlists utilizing logic synthesis tools, outlining gates and their interconnections. Subsequently, place and route algorithms are used for the placement of gates and routing of interconnections. However, this translation incurs information loss from a reverse engineering standpoint (from netlist to RTL), particularly in terms of module boundary and hierarchy information [10, 11]. Moreover, various optimizations are implemented to achieve predefined goals like improved area or timing. Consequently, reverse engineers target their data to recover essential high-level design information [12–17].

A comprehensive case study is presented to provide an exhaustive review of the intersection between machine learning (ML) techniques and hardware security, specifically focusing on the detection and mitigation of hardware Trojan attacks using supervised learning techniques have been prominently discussed for their high accuracy in classifying known Trojan signatures [12]. In addition, recent advancements have introduced efficient ML-based HT detection methods that leverage statistical and structural features for improved classification accuracy [13], while golden-free variability-aware detection frameworks based on PUF signatures have further demonstrated the effectiveness of ML-driven approaches in scenarios lacking a trusted reference model [14]. A novel technique that enhances the efficiency of extracting FSMs is also proposed in the past that is very helpful in understanding the behaviour of digital circuits [15]. Building upon previous work, the research work explores into the extraction of functional modules from flattened gate-level netlists. The research expands understanding of identifying and categorizing functional components within digital circuits, offering valuable insights for reverse engineering practitioners [16]. The behavioral pattern mining-based reverse engineering circuits technique is also explored in the past. It provides the application of pattern mining in reconstructing circuit behaviors, contributing to the advancement of effective reverse engineering methodologies [17, 18]. A template-based solution is also proposed to the challenges posed by circuit complexity, contributing to the arsenal of tools available for reverse engineers [19]. Previous approaches encounter difficulties in fully recovering module boundaries and hierarchy from gate-level netlists due to information loss during synthesis. To address this, the proposed work introduces an innovative method for more accurate and exact locations of HT inside complex ICs.

3 Methodology

Recently, HT detections have made substantial use of machine learning approaches. Among these, GNNs have shown promising results due to their ability to effectively model the complex interconnections within a circuit represented as a netlist [20]. At the gate-level netlist stage of the IC design flow, the circuit is represented as a collection of interconnected logic elements such as gates, flip-flops, and multiplexers. Since, this representation is non-Euclidean, the proposed model converts the netlist into a structural graph, where nodes are corresponded to logic components and edges capture signal dependencies. This transformation preserves the essential structural information of the design.

The proposed framework HW2GRAPH is designed to transform gate-level netlist files into graph-based representations that can be effectively utilized for machine learning–driven security analysis (refer Algorithm 1). To prepare this representation for automated learning, the netlist is first pre-processed. Multiple Verilog source files are flattened into a single file to remove hierarchy-related fragmentation and the top module is automatically identified to define the circuit's functional boundary. This ensures that all module instances, interconnections, and signal dependencies are preserved in a unified design view.

The flattened netlist is then parsed using PyVerilog [21], which converts the design into a structural representation that can be traversed systematically. From this parsed output, the framework constructs a graph where nodes correspond to logic gates and sequential elements, while edges represent the signal dependencies between them. Primary inputs and outputs are modeled as special nodes to anchor the flow of data across the circuit. The resulting graph captures both the structural and functional properties of the design, enabling circuits of different sizes and complexities to be represented in a consistent manner. The generated graph is then exported into JSON format and converted into a NetworkX object using the NetworkX framework of python [22], making it directly compatible with widely used geometric deep learning libraries. Once the structural graph has been generated and stored as a NetworkX object, the next step is to enrich it with descriptive node-level features to prepare it for supervised learning tasks such as hardware Trojan detection.

Each node is characterized using three categories. (i) Structural features include one-hot encoded gate types (e.g., AND, OR, XOR, NOT, DFF, MUX), fan-in and fan-out counts, and the logic level relative to primary inputs and outputs. (ii) Topological features describe the node's position in the global graph, incorporating metrics such as degree, clustering coefficient, and shortest path lengths to I/O nodes. (iii) Neighbourhood features capture the local context by aggregating information from the first and second nearest neighbours, including gate-type distributions, sequential-to-combinational ratios, and local connectivity density. Together, these features capture both low-level attributes of gates and higher-order relationships within the design, which are crucial for identifying malicious modifications.

Once the graph has been constructed in NetworkX and enriched with structural, topological, and neighbourhood features, the next stage converts this non-Euclidean representation into compact Euclidean embeddings that can be directly used for machine learning. The transition begins by exporting the NetworkX object into a format compatible with geometric deep learning libraries such as PyTorch Geometric [23]. In this

conversion, the graph is expressed as an adjacency structure (edge index or adjacency matrix) and a node-feature matrix, where each row corresponds to a gate and each column encodes one of its extracted features. Together, these two matrices capture both the circuit topology and the descriptive attributes of each node, forming the input for the embedding process.

GRAPH2VEC leverages Graph Neural Networks (GNNs) to learn vector representations of the hardware graph (refer Algorithm 1). The process relies on iterative message passing, where each node updates its representation by aggregating information from its neighbours. In this way, a node's embedding becomes a function not only of its own structural features but also of the features of the gates it connects to, and eventually the wider subcircuit to whom it belongs. Stacking multiple GNN layers allows information to propagate across larger neighbourhoods, enabling the model to capture both local structural patterns and global circuit-level dependencies. To transition from node-level embeddings to a fixed-length representation of the entire netlist, graph pooling and readout functions are applied. Pooling operations, often attention-based, identify and prioritize the most informative nodes and subgraphs, while readout functions max aggregation condense the node embeddings into a single graph-level embedding vector. This embedding resides in a Euclidean space, where structurally similar designs cluster together while clean and Trojan-infected circuits are well-separated.

Subsequently, the model is trained using labelled datasets containing both Trojan-free and Trojan- infected netlists. Finally, the performance is evaluated on the trained model. Matrices such as accuracy, precision, recall, and F1-score are generally used for assessing the efficiency of machine learning models in Trojan detections. The GNN methodology holds the potential to complement existing detection techniques and pave the way for more robust and scalable solutions in safeguarding electronic systems against hardware-level threats.

In this work, three cases are examined. Case I uses a decision tree with PCA. It has been used as reference model for the validation against formal verification method. Case II applies GNN-based graph-to-graph classification to distinguish clean from infected netlists enabling coarse-grained detection at the sub-graph level where a Trojan may reside inside the circuit. Case III performs GNN-based node classification for fine-grained localization of Trojan-infected gates at node level.

4 Dataset and model

The proposed model employs a set of benchmarks comprising gate-level netlists taken from Trust-Hub [24]. In order to make datasets more generalized, additional netlists of new designs are also generated and converted to netlist using Yosys [25]. These netlists contain a variety of HTs, such as Trigger-based Trojans (95 types), always on Trojans (11 types), Payload-based Trojans (154 types) in combinational and sequential logic [26, 27]. The dataset is composed of total 1025 circuits including 735 designs from the Trust hub (120 clean and 615 Trojan-inserted variants) and additional 290 custom made non-infected designs converted to netlist from Yosys. In total, the combined benchmark set comprises 410 clean designs and 615 Trojan-inserted designs, providing a balanced and diverse dataset for training and evaluation. The dataset is curated from Trust-Hub and serves as a widely used benchmark for HT detection, featuring diverse circuits, including AES (29 designs) (nodes vary from 10756to 13438), RS232 (9 designs)

(nodes vary from 8,742 to 10210), memctrl (5 designs) (nodes vary from 1842 to 2106), PIC16F84 (9 designs) (nodes vary from 12487 to 15326), WB_CONMAX (6 designs) (nodes vary from 39214 to 46832), a 32-bitRISC-Vprocessor (80206 nodes), NoCRouter (20727nodes), Chipyard SoC (3 designs) (nodes vary from 28765 to 32000) (The Chipyard SoC design is used in Industry grade IC UC Berkeley framework, widely used for real silicon prototyping [28]), Rocket RISC V family (24000 nodes), Open Cores(4 designs) (nodes vary from 52000 to 56000) (Open Cores is an industry-grade well-tested IP cores for SoC design [29]), VexRiscv (8 designs) (nodes vary from 480000 to 510000) (VexRiscv is a heavily optimized industry grade RISC V core, FPGA and ASIC validated [30]) and many more combinational and sequential circuits that are closest to industry grade ICs. These circuits range from small controllers to larger system-on-chip (SoC) designs, covering both combinational and sequential logic. Each circuit has multiple versions (clean and Trojan-infected) ensuring a comprehensive evaluation framework. In the Trust-Hub dataset, the benchmark documentation explicitly specifies the location and gate-level structure of each inserted Trojan. During netlist parsing, these gates and their associated fan-in/fan-out connections are mapped to the corresponding nodes in the extracted graph representation and annotated as Trojan-infected, while all remaining gates are labeled as Trojan Free. These Trojans employ three triggering mechanisms: input data-triggered HTs (activated by specific input patterns), normally open Trojans (active throughout operation), and state-based HTs (triggered by predefined internal states, such as a counter value). Beyond Trust-Hub, the present work expands the benchmark with 290 Verilog-based clean designs that includes 32-bit fixed-point adder (998 nodes), 32-bit fixed-point multipliers with 1–3 pipeline stages (2105, 2123, 2187 nodes), 32-bit floating-point multiplier (2646 nodes), 32-bit floating-point adder (1620 nodes) derived feature-extracted data for ML-based classification [31]. The details of individual machine learning models are provided in the next section.

Case-I: Confirmation of HTs using machine learning

Case-I presents a structured workflow for detecting HTs in digital circuits using an ML technique, specifically focusing on a decision tree algorithm [18] (refer Fig. 2 and Algorithm 1 for further details). In Yosys, the RTL description undergoes synthesis and optimization to generate a gate-level netlist. The process begins with frontend parsing, where Yosys reads the Verilog or VHDL code and converts it into an intermediate representation (IR). Next, logic synthesis is performed, mapping high-level constructs into primitive gates such as AND, OR, and flip-flops. The technology mapping stage then optimizes the design by selecting appropriate standard cells based on the synthesis library. Finally, the flattening and netlist export stage produces a structural gate-level netlist, which is then used for graph-based Trojan detection, where gates become nodes and connections form edges in the extracted circuit graph. The next step involves preparing the data for machine learning. Each node and edge in the graph are encoded with labels to facilitate further analysis. The encoded data undergoes vectorization and transforming it into a numerical format suitable for machine learning models. The core of the process employs a decision tree algorithm. It is used for analysing the vectorized data and recognizes patterns or anomalies suggestive of HTs in binary format. This approach facilitates the identification of potential HTs by comparing the node index. The Case-I is considered as reference machine learning model, which is taken from [18]. In [18], 426

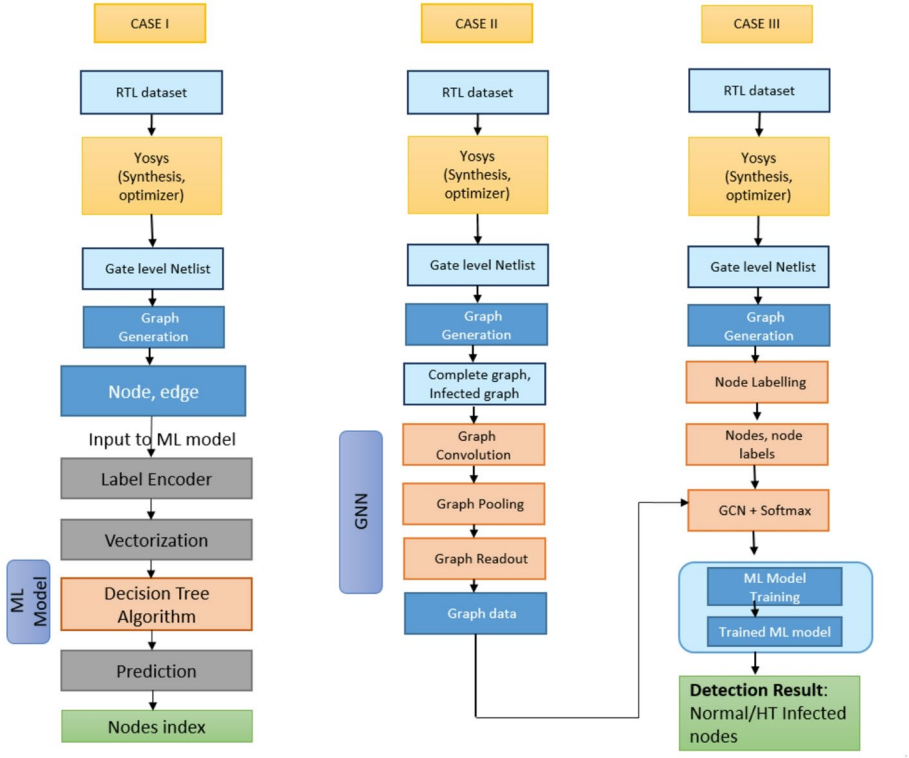


Fig. 2 The proposed methodology

datasets are considered, taken from TrustHub [24]; however, in the present Case-1, the data set is expanded (includes total 1025 circuits). The model is also validated against the formal verification method [35]. It has been observed that all the infected circuits of [35] (consist of AES and Basic RSA) are able to detect by the proposed model of Case-1.

Algorithm 1: Decision Tree for the confirmation of Trojan in netlist

Input: Gate level Netlist files (Verilog) of circuits.*v*
Output: Decision (*D*) = Trojan Infected node or Trojan Free node for each circuit .*v* do;

```
g←HW2GRAPH (.v);
hg←GRAPH2VEC (g);
ŷ← DecisionTree(hg);
If ŷ==0 then
    Return NON_TROJAN;
else
   return TROJAN
```

The decision tree method effectively reduces the dimensionality of the feature space, allowing for more accurate comparisons between nodes. However, this approach has limitations. Decision trees are suitable for simple, binary classification problems. However, it is not feasible to pinpoint the exact location of HTs in terms of sub-graphs and nodes using decision tree approach. In order to overcome this, GNNs based techniques are explored. GNN is more robust approach, capturing complex relationships and improving detection accuracy in intricate Trojan designs.

Case-II: GNN model using graph to graph comparison

To address the limitations of Case-I, a graph-to-graph classification approach using a Graph Neural Network (GNN) model is introduced as Case-II. This method enables

differentiation between normal and Trojan-infected circuit designs by leveraging the structural properties of graphs. The GNN model can effectively capture complex patterns and relationships within circuit designs, making it well-suited for detecting sophisticated Trojans.

Case II illustrates a workflow for generating graph data from RTL datasets using the Neural GNN model (refer Fig. 2 and Algorithm 2 for further details). Initially, RTL datasets are synthesized and optimized using Yosys, producing a gate-level netlist. This netlist undergoes graph generation to form a complete graph, including infected graph data, which represents potential errors or modifications. The methodology involves transferring a given netlist into graph representation where nodes correspond to logic gates and edges define them the interconnects. A database of identified infected graph containing Trojan patterns is created for comparative analysis.

To identify Trojan location, the netlist graph is systematically compared against infected graphs using graph intersection technique. When a subgraph within the netlist exhibits a significant overlap with a non-infected graph, it is flagged as potential Trojan affected region. The nodes involved in this intersection are extended and mapped back to the original netlist for pin point the exact location. The GNN model then processes the graph through three main stages, i.e. (a) graph convolution, graph pooling, and (c) graph readout. Graph convolution extracts feature from the graph, graph pooling reduces the graph's complexity while preserving essential information, and graph readout generates the final graph data output. These labelled graphs are passed through a Graph Convolutional Network (GCN) with a Softmax activation function to extract features and perform node classification.

The foundation of methodology lies in formulating the HT detection problem as a binary classification task, where each circuit instance is classified as either benign or Trojan-infected. A comprehensive dataset comprising both Trojan free and Trojan-infected circuits is then constructed, ensuring a diverse and representative training corpus.

A database of known Trojan-infected graphs is created, and detection is performed by comparing the new netlist graph against this database using graph intersection. If subgraph overlaps are found, the suspicious regions are mapped back to the original netlist. This approach leverages the graph's structural information and the ML models' learning capabilities to efficiently detect and classify potential HTs within the dataset. Thus, the model demonstrates the efficacy of the deep learning-based approach in accurately detecting HTs. Implementation details are provided in Algorithm 2.

Subsequently, nearest neighbour (NN) techniques (i.e., 1st and the 2nd NNs) are integrated with the base GNN model for further improving the accuracy of the model. By taking into account each node's immediate neighbour, i.e., 1st nearest and the next closest to the 1st nearest neighbour, i.e., 2nd nearest, the model enhances its capability to detect HTs. This layered neighbour analysis provides additional context, enabling more accurate identification of Trojan related irregularities. Furthermore, the proposed approach is even more effective for large scale circuits where exhaustive search technique is computationally impractical.

Algorithm 2: Trojan Detection using GNN based Graph-to-Graph Classification

```
Input: Gate-level Netlist files (Verilog) of circuits (.v)
Output: Trojan Infected Node or Trojan-Free Node with Nearest Neighbour Analysis

def Trojan_location_detection(netlist_files):
  for each circuit.v in netlist_files do:
g ← HW2GRAPH (.v)
hg ← GRAPH2VEC (g)
ŷ ← GCN (hg)
          # Generate infected graph representation
infected_graphs = generate_infected_graphs()

    # Graph intersection to detect Trojan affected subgraph
intersection_graph = graph_intersection (g, infected_graphs)

    if intersection_graph:
      # identify specific nodes involved in intersection
Trojan_nodes = extract_trojan_nodes(intersection_graph)

      # Map detected nodes to original netlist
Trojan_locations = map_to_netlist (Trojan_nodes, circuit)

      return f"TROJAN detected in nodes: {Trojan_locations}"

    return "NON_TROJAN"
  for node in g.nodes:
    NN1 ← find_nearest_neighbour (node, level=1)
    NN2 ← find_nearest_neighbour (NN1, level=2)
f_i ← extract_features (node, NN1, NN2)
y_node = GCN (f_i)

    if y_node == 'TROJAN':
      return f"TROJAN detected in node: {node}"

return NON_TROJAN
```

Case-II enhances the capabilities as identification of infected sub-graphs in the circuit compared to Case–I where only confirmation of HTs was insured. However, further improvement in Case-II is carried out where identification of individual compromised node locations using GNN model is proposed within a circuit design.

Case-III: GNN model using node classification

Case-III enhances detection by applying GNN-based node classification, enabling precise identification of individual compromised nodes and their locations. This approach leverages the graph structure of a circuit design to predict node labels (normal or infected). By applying this method at the node level, the extent and location of compromise within a circuit design can be gained.

Case-III presents a detailed workflow for detecting HT in RTL datasets using a combination of graph-based methods and machine learning techniques. The process begins with the RTL dataset, which is synthesized and optimized using Yosys, resulting in a gate-level netlist. This netlist is then converted into a graph structure during the graph generation phase. The next step involves node labelling, where each node, edge, and node label within the graph is identified and annotated. Figure 3 illustrates the proposed GNN-based node classification framework, highlighting how individual gates in the netlist graph are classified as Trojan-infected or Trojan-free for fine-grained localization. These labelled graphs are processed through a GCN followed by a Softmax activation function to extract features and classify the nodes. The processed graph data, including nodes, edges, and their labels, is then used for training a ML model. The trained ML model is subsequently employed to detect HT-infected nodes, providing detection results that distinguish between normal and HT-infected nodes. This method leverages

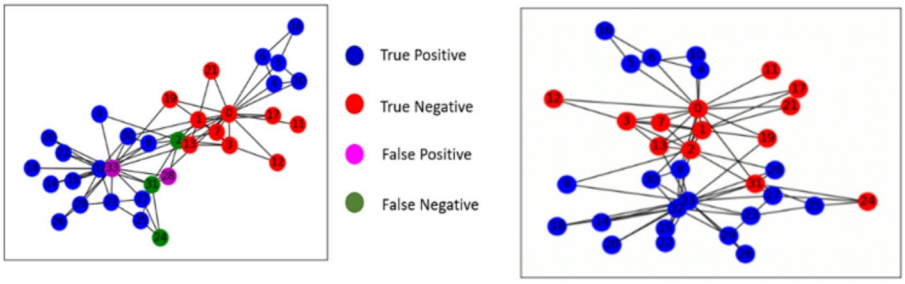


Fig. 3 Node classification in netlist

the structural information of the graph and the learning capability of ML models to effectively identify and classify potential hardware Trojans in the dataset. Implementation details are provided in Algorithm 3. Additionally, the model is integrated with 1 st and 2nd NN approaches, as discussed in Case-II.

Algorithm 3: GNN Model for Node-to-Node Classification

```
Input: Gate-level Netlist files (Verilog) of circuits (.v)
Output: Trojan Infected Node(s) or Trojan-Free Node(s)
def trojan_detection_node_classification(netlist_files):
infected_nodes = []  # To store infected nodes
for each circuit.v in netlist_files do:
g← HW2GRAPH(.v)
hg← GRAPH2VEC(g)
 ŷ← GCN(hg)

for node in g.nodes:
      ŷ_node = GCN_node_classification(node, hg)

      if ŷ_node == 1:
      infected_nodes.append(node)

if infected_nodes:
      post_process_infected_nodes(infected_nodes, circuit)
        returnf"TROJAN Detected in nodes: {infected_nodes}"

return "NON_TROJAN"

# Post-Processing Function
def post_process_infected_nodes(infected_nodes, circuit):

/*This function processes the identified infected nodes by mapping them
back to the original circuit netlist and providing a clear output.

Parameters: infected_nodes (list): A list of detected compromised nodes.
circuit (object): The circuit netlist where nodes need to be verified.

Returns: None (Prints the identified infected nodes with their
corresponding netlist locations).
*/
for node in infected_nodes:
node_in_netlist = find_node_in_netlist(node, circuit)
print(f"Infected node found in netlist:{node_in_netlist}")
```

5 Results and discussions

All the experiments are carried out on a Workstation with an Intel I Core I i7-4770 CPU, 16 GB of RAM, and Ubuntu 20.04.1 as the operating system.

Case-I: Confirmation of HTs using machine learning model

The study compared the performance of HT detection with and without PCA across various input shapes, as shown in Table 1. For input shapes of 226, 808, and 1025, the

Table 1 Classification performance comparison with and without PCA

Parameters	Without PCAs			With PCA			With t-SNE			With UMAP		
Input shape	226	808	1025	226	808	1025	226	808	1025	226	808	1025
With Trojan	128	593	711	128	593	711	128	593	711	128	593	711
Without Trojan	98	215	314	98	215	314	98	215	314	98	215	314
Accuracy	35%	41%	41%	42%	98.3%	97.54%	45%	82%	79%	44%	87%	85%

dataset comprised 128, 593, and711 samples with Trojans and 98, 215, and 314 samples without Trojans, respectively. The data is split with an 80%:20% between training and test data. Without PCA, the accuracy achieved was 35%, 41%, and 41% for the respective input shapes. In contrast, applying PCA resulted in significant improvements, yielding accuracies of 42%, 98.3%, and 97.54%. These findings indicate that PCA effectively enhances the model's performance, particularly for larger input shapes, by reducing dimensionality while preserving critical features essential for accurate Trojan detection. Four random sample's outputs are shown in Fig. 4, where three cases have Trojan and one is without a Trojan. It can be seen that the model predicts the Trojan accurately in all the four cases. However, in one case (i.e., in Fig. 4(c)), the node index is not predicted correctly. The results of PCA are also compared with other techniques, i.e. t-SNE [32] and UMAP [33]. For comparison, t-SNE 10 dimensions are selected, since reducing to 2D caused excessive information loss, while 10D preserved local neighbourhood structure more effectively [32]. For Uniform Manifold Approximation and Projection (UMAP), 30 dimensions are selected. It balances local and global structure preservation but still proved less stable and accurate than PCA [33]. This selection ensures minimal loss of critical features while improving processing speed. Table 1 presents a comparative evaluation of dimensionality reduction methods (PCA, t-SNE, and UMAP) for HT detection in gate-level netlists. The t-SNE achieves moderate improvements 79%, showing that while it preserves local neighbourhood structure, it suffers from instability and scalability issues in higher dimensions. UMAP performs better than t-SNE, with accuracy 85%, due to its balance between local and global structure preservation, however, it still remains less effective than PCA's performance. These results demonstrate that PCA not only ensures computational efficiency but also provides more robust and stable accuracy compared to other non-linear methods in the context of large-scale netlist-based HT detection.

Case II: GNN model using graph to graph classification

In this section, a GNN model with graph-to-graph classification is presented. Firstly, the details of various nodes used in this study are explained. Figure 5 compares a network with (a) true structure and (b) a predicted structure, where red nodes are infected outputs and blue nodes are features. Green nodes in the predicted graph indicate false negatives, meaning the model missed these connections during prediction. Figure 5(c) isolates the infected output nodes, highlighting the nodes in the network that are predicted as compromised. This is done for the train dataset. Figure 6 provides a more detailed view of a graph-based HT detection for the test data set, highlighting three categories of nodes. Red nodes represent target nodes, blue nodes indicate infected nodes, green nodes indicate false negative nodes and yellow nodes denote the 1 st NN Nodes. This visualization emphasizes the use of the NN algorithm to enhance Trojan detection by expanding the scope of analysis beyond just the infected nodes. The inclusion of 1 st NN nodes suggests a broader capture of potentially compromised areas in the hardware, further improving the accuracy and coverage of the detection model. This model reinforces the value of the NN algorithm in augmenting graph-based detection models by identifying nodes closely connected to infected ones, increasing the likelihood of uncovering hidden or adjacent threats within the circuit.

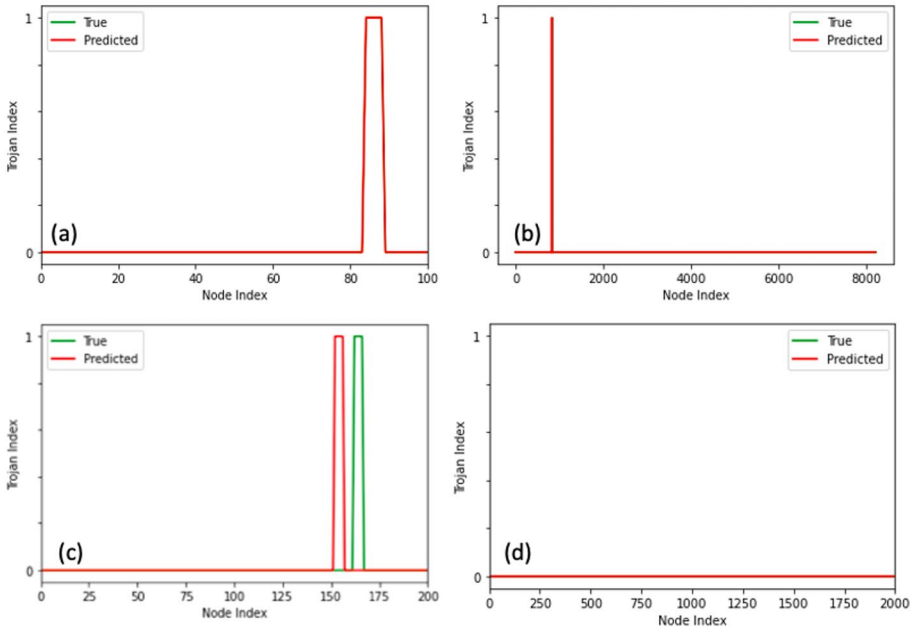


Fig. 4 HT detection results using the decision tree model for four randomly selected test cases: (a) Trojan-infected circuit correctly classified as infected, (b) another Trojan-infected circuit correctly detected, (c) Trojan-infected circuit where the presence of HT is correctly identified but the exact node index is misclassified, and (d) Trojan-free circuit correctly classified as non-infected.

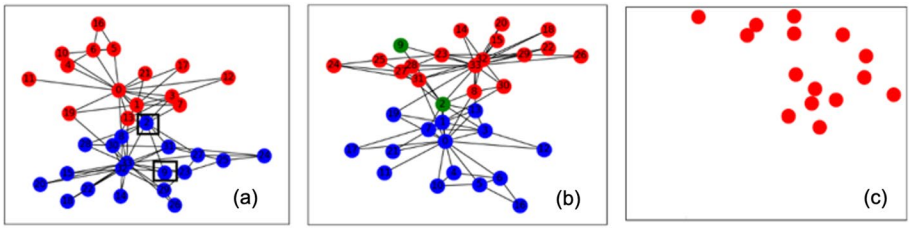


Fig. 5 GNN model prediction for training data set (**a**) true graph, (**b**) predicted graph, and (**c**) infected nodes (target nodes (Red), infected nodes (blue), and false negative nodes (green))

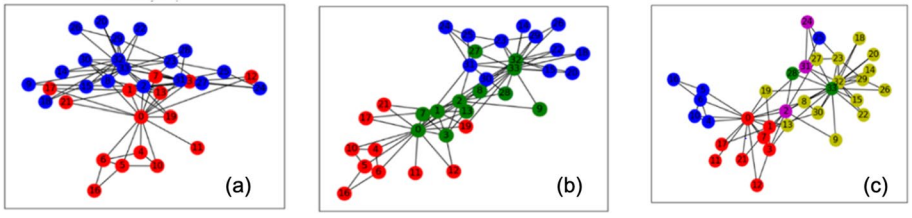


Fig. 6 GNN model prediction for the testing data set including 1 st NN nodes (**a**) true graph, (**b**) predicted graph, and (**c**) infected nodes (target nodes (Red), infected nodes (blue), False Negative nodes (green), and 1 st NN nodes (yellow))

The GNN model with graph-to-graph shows an accuracy of 62.8%, as shown in Table 2. Subsequently, 10 random benchmarks have been considered for further data analyses. It is observed that the average and maximum code coverage of GNN model average for finding HTs in the circuits are 3.54% and 6.7% for the 10 benchmarks, respectively (refer Fig. 7; Table 2 for further details). The 10 benchmarks consist of AES (27 designs), C432 (23 designs), C499 (22 designs), C880 (27 designs), PIC16F84 (4 designs), RS232

Table 2 Accuracy and code coverage for graph-to-graph (G-G) prediction model for 10 benchmarks (Case-II)

	Accuracy	Average code coverage	Maximum code coverage	Average detection time (s)	Time saving for HT detection
		(%)	(%)		(%)
GNN G-G	62.8	3.5	6.7	0.805[$]	~93%
GNN G-G with 1st NN	73.2	5.0	7.8	1.235 [$] (0.430*)	~92%
GNN G-G with 2nd NN	97.7	32.1	50.0	1.537[$] (0.302[#])	~50%

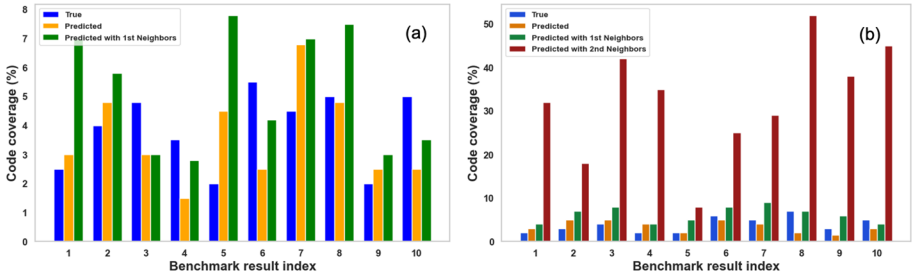


Fig. 7 Graph-to-Graph prediction model (Case II), (**a**) Prediction with 1 st NN and (**b**) Prediction with 2nd NN

(10 designs), C3540 (14 designs), S1238 (16 designs), S1423 (8 designs), and S953 (13 designs).

Next, both the 1 st and 2nd NN methods have been used to check the improvements in the accuracy of the basic GNN graph-to-graph model, as shown in Table 2. Result shows that the 1 st NN method improves the model accuracy from 62.8% to 73.2%. However, it has to be noted that the average code coverage and maximum code coverage for finding HTs in the circuits are 5.0% and 7.8% for these 10 benchmarks, respectively (refer Fig. 7; Table 2 for further details). It leads to approximate 92% time saving compared to manual identification of HTs in the circuits. When comparing the model's performance with the 2nd NN approach, the GNN model showed an improved prediction accuracy of 97.7%. The average and maximum code coverage for finding HTs in the circuits using GNN integrated with 2nd NN approach for 10 benchmark samples are 32.1% and 50.0%, respectively. The 2nd NN approach leads to approximate 50% time saving compared to manual identification of HTs in the circuits.

These results highlight that while the GNN model shows promising results, the integration of NN approach leads to superior performance in terms of accuracy with significant amount of time saving while identifying the exact locations of the HTs inside the circuits. Also, it can be noted that the average time for model to predict the HTs for 10 benchmarks (total 164 designs) is very less (refer Table 2 for further details). Hence, the GNN graph-to-graph model along with the NN approach (either 1 st or 2nd) can be used intelligently to exactly pin-point the HT locations in a large complex IC circuit. Also, the interpretability is provided in the present work by identifying the infected subgraphs along with their nearest neighbours, and then mapping these back to the original netlist (sub-graphs) nodes for inspection. Figures 5, 6, 7 and 8 illustrate visualizations of these predicted infected regions, enabling designers to visually trace and validate the results.

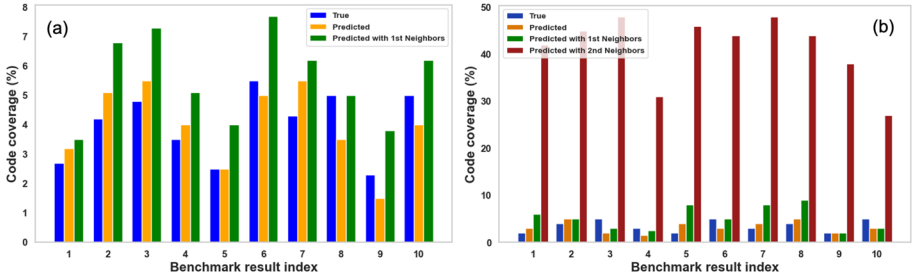


Fig. 8 Node-Node prediction model (Case III), (**a**) Prediction with 1 st NN and (**b**) Prediction with 2nd NN

Table 3 Performance of proposed model for graph-to-graph Trojan detection (Case II)

Input layer	Hidden layers	Optimizer	No of epochs	Output layer	Test accuracy (%)	Precision (%)	F1-score (%)	Accuracy (%) with NN (known)	
								1st NN	2nd NN
Adjacency matrix of DFG	12	Adam	200	Infected Nodes	56.7	58.3	57.5	72.0	83.0
Adjacency matrix of DFG	**16**	**Adam**	**200**	**Infected Nodes**	**62.8**	**64.3**	**63.6**	**73.2**	**91.7**
Adjacency matrix of DFG	16	AdamW	200	Infected Nodes	59.3	60.3	59.4	66.8	84.9
Adjacency matrix of DFG	16	Adam	250	Infected Nodes	61.6	62.6	61	72.4	90.4
Adjacency matrix of DFG	16	Adamax	250	Infected Nodes	58.5	59	59	69.8	85.9

Details of ML parameters used in the GNN graph-to-graph classification approach are discussed next. It includes input layer, hidden layers, classification, optimizer, learning rate, number of epochs, loss function, and output layers, as shown in Table 3. A large number of parametric studies are carried out and finally, model with 16 hidden layers, Softmax classification, Adam optimizer, learning rate of 0.001 with 200 epochs, and loss function of cross entropy is considered. The proposed GNN graph-to-graph model shows the test accuracy of 62.8%, precision of 64.3%, and F1-score of 63.6%. The accuracy further improves to 73.2% and 91.7% for known test datasets with 1 st NN and 2nd NN, respectively. The GNN graph-to-graph model with NN is also tested against unknown datasets, for which accuracy of 92.7% is achieved, as shown in Table 2.

Also, in order to ensure that the selected hyperparameters are not arbitrary, we build upon our earlier investigations in [20] and [27]. In [20], a sensitivity analysis was carried out at the RTL level, examining the effect of learning rate, model depth, and hidden size on GNN-based Trojan detection and in [27], a comparable analysis was performed at the gate-level netlist abstraction. The insights gained from these studies guided the present configurations, ensuring robustness of the chosen hyperparameters to dataset and design variations.

Case-III: GNN model using node-to-node classification

The performance of Case-III is shown in Fig. 8. In the evaluation of the GNN model using node-to-node classification, the model achieved a prediction accuracy of 79.8%. The GNN model's average code coverage for finding HTs in the circuit is 3.9%, and its

Table 4 Accuracy and code coverage for node-to-node prediction model for 10 benchmarks (Case-III)

	Accuracy	Average code coverage (%)	Maximum code coverage (%)	Average detection time (s)	Time saving for HT detection (%)
GNN N-N	79.8	3.9	5.4	0.840[$]	~94%
GNN N-N with 1 st NN	93.0	5.5	7.7	1.35[$] (0.510[*])	~92%
GNN N-N with 2nd NN	97.7	32	50.0	1.86[$] (0.510[#])	~50%

Table 5 Performance of proposed model for node-to-node Trojan detection (Case III)

Input layer	Hidden layers	Optimizer	No of epochs	Output layer	Test accuracy (%)	Precision (%)	F1-score (%)	Accuracy (%) with NN (known)	
								1st NN	2nd NN
Adjacency matrix of DFG	10	Adam	250	Node Labels	73.3	72.6	73.8	83	86.2
Adjacency matrix of DFG	12	Adam	250	Node Labels	79.8	81	80	93	97.7
Adjacency matrix of DFG	12	AdamW	250	Node Labels	74.6	76.3	75	91.8	96.3

maximum code coverage is 5.4% for the 10 benchmarks, as shown in Fig. 8; Table 4. The 1 st NN method leads to an improve in accuracy with 93.0%. Th average and maximum code coverage for finding HTs in the circuit for the 1 st NN model is 5.5% and 7.7%, respectively. It leads to approximate 92% time saving compared to manual identification of HTs in the circuits. When compared to the 2nd NN approach, which significantly outperformed with an accuracy of 97.7%. The average code coverage for finding HTs in the circuit is 32.1% and maximum code coverage of 50.0% is obtained for the 2nd NN approach. It leads to approximate 50% time saving compared to manual identification of HTs in the circuits. These results highlight that while the GNN model using node-to-node shows promising results, the integration of nearest neighbour approaches; deliver superior performance in terms of accuracy. Also, it can be noted that the average time for model to predict the HTs for 10 benchmarks (total 164 designs) is very less (refer Table 4 for further details). Hence, the GNN node-to-node model along with the NN approach (either 1 st or 2nd) can be used intelligently to exactly pin-point the HT locations in a large complex IC circuit. It has also to be noted that the GNN model with node-to-node (Case III) has better accuracy compared to GNN model with graph- to-graph (Case II) without integration of NN approach and also with the 1 st NN approach.

Table 5 shows the parametric studies carried out for the GNN model with node-to-node classification. It can be seen that the adjacent matrix of DFG is used as an input layer along with various combinations of hidden layers, RELU and softmax classification. Different optimizers, i.e., Adam and AdamW, have been studied. The performance of the basic GNN model with node-to-node, with the 1 st NN and 2nd NN has been evaluated. It can be seen that the NN along with a basic GNN model with node-to-node classification, helps to improve the accuracy, surpassing 90% in most cases. The highest accuracy achieved was 98.5% with 12 hidden layers, Softmax output, Adam optimizer, learning rate of 0.001, 250 epochs, and Categorical cross-entropy loss function. The

Adam optimizer consistently outperformed AdamW, demonstrating its effectiveness for this task.

Next, few new untested datasets are taken from TrustHub [24] and tested using the proposed model without any further changes in it, as shown in Table 6. It can be seen from the Table 6 that the proposed model maintains consistently high accuracy (>99%) across all benchmarks. AES-T-101 achieves the highest precision (99.8%), indicating minimal false positives, while RS232-T600 shows slightly lower precision (97.5%) but balanced recall. RC5 demonstrates the fastest inference time (37 ms) due to its smaller node count, whereas larger designs such as AES-T101 and RS232-T600 still achieve efficient runtimes within a few hundred milliseconds. Overall, the results highlight the robustness of the framework in delivering both high detection accuracy and practical scalability across diverse benchmarks.

The comparison of various state-of-the-art hardware Trojan detection models are also presented in Table 7. It highlights distinct trade-offs in detection accuracy, localization capability, and computational efficiency. Traditional models such as GNN4TJ [26], GNN [27], DFF [34] and FV [35], demonstrate strong detection performance; however, lack localization abilities, limiting their effectiveness in pinpointing compromised nodes. Methods like FastText [36] and Decision Tree [37] incorporate localization; however, suffer from lower recall or structural limitations. Advanced graph-based techniques, such as GCN [38], GNN [39] and node-wise detection models [40], offer improved node extraction and edge-level insights, enhancing Trojan identification. The proposed model outperforms prior methods by integrating structural netlist-level features in GNN with the nearest-neighbour (NN) method, which strengthens discrimination between Trojan-infected and benign nodes. It is also to be noted that the dataset used in the present study is much larger compared to the previous studies. This ensures the robustness and superior performance of the proposed models.

6 Conclusions

This proposed work introduces a robust and scalable framework for HT localization in gate-level netlists, integrating nearest-neighbour analysis with ML to balance accuracy, scalability, and interpretability. By leveraging structural netlist features, path-retrace algorithms, and efficient dimensionality reduction, the framework effectively identifies malicious nets and pinpoints their locations with superior performance. Across three experimental cases, the methodology demonstrated consistent improvements: Case I showed that integrating PCA with decision trees enhanced classification of infected and non-infected dataset; Case II and Case III employed graph neural networks (GNNs) for graph-to-graph and node-to-node classifications, where the integration of the nearest-neighbour (2nd NN) method achieved improved performance with 97.7% accuracy, outperforming standalone GNN models. Extensive evaluations on diverse benchmarks,

Table 6 Transferability test on untested designs

Metric	All	AES-T101	RS232-T600	RC5
Accuracy	99.6%	99.8%	99.8%	99.2%
F1-score	93.1%	93.2%	92.2%	93.1%
Precision	99.0%	99.8%	97.5%	99.4%
Recall	88.0%	88.0%	87.9%	88.0%
Number of Nodes	2000–14,000	13,438	10,212	2106
Time	436 ms	222 ms	162 ms	37 ms

Table 7 Comparison with other state-of-art models

Method	Dataset used	No. of data samples	Detection	Localization	Precision	Recall (TPR)	F1-Score	Node extraction	Edges extraction	Nearest neighbour
GNN4TJ[26]	Trust-HUB	100	✓	✗	92.3%	96.6%	94.0%	✓	✓	✗
GNN[27]	Trust-HUB+CustomHDL	422	✓	✗	96.6%	100.0%	98.4%	✓	✓	✗
DFF[34]	Trust-HUB+Opencores+CustomHDL	98	✓	✗	97.8%	93.8%	95.5%	✓	✓	✗
FV[35]	Trust-HUB	28	✓	✗	-	-	-	✗	✗	✗
Fast-Text[36]	Trust-HUB	52	✓	✓	97.9%	97.8%	97.5%	✓	✓	✗
Decision-Tree[37]			✓	✓	100.0%	78.8%	88.1%	✓	✓	✗
GCN[38]	Trust-HUB	38	✓	✓	98.9%	88.4%	93.1%	✓	✓	✗
GNN[39]	Trust-HUB	132	✓	✓	94.5%	99.1%	96.7%	✓	✓	✗
Node-wiseHTdetection[40]	Trust-HUB	24	✓	✓	95.0%	98.0%	96.4%	✓	✓	✓
Proposed Model	Trust-HUB+Hardware Custom HDL	1025	✓	✓	98.5%	99.1%	96.7%	✓	✓	✓

including AES, RS232, and RISC-V, validated the model's generalization capability and transferability to industry grade designs. Furthermore, the framework reduced manual inspection effort drastically up to ~94% compared to GNN based node-to-node machine learning method, underscoring its practical value for real-world IC security. Beyond surpassing state-of-the-art detection and localization methods, this work establishes a pathway toward industry-grade deployment of ML-driven HT detection, while highlighting the broader potential of NN–GNN integration for anomaly detection in other high-dimensional domains.

Author contributions
Anindita Chattopadhyay and Siddharth Bisariya has contributed to methodology, formal analysis, data generation, validation and writing original draft and Vijay Kumar Sutrakar has done the conceptualization, methodology, supervision, writing original draft, writeup review and re-editing.

Funding
This research has not been supported by any funding from any Institution.

Data availability
dataset is available at [https://github.com/aninchatto/Fast-and-Accurate-Identification-of-Hardware-Trojan-Locations-in-Gate-Level-Netlist-using-ML](https:/github.com/aninchatto/Fast-and-Accurate-Identification-of-Hardware-Trojan-Locations-in-Gate-Level-Netlist-using-ML).

Declarations

Ethics approval and consent to participate
This research does not involve any humans or living beings and does not harm the environment, either directly or indirectly. This article does not contain any studies with human participants or animals performed by any of the authors.

Consent for publication
We hereby give our consent for [Springer Nature] to publish the work entitled "Fast and Accurate Identification of Hardware Trojan Locations in Gate-Level Netlist using Nearest Neighbour Approach integrated with Machine Learning Technique" in Discover Electronics or on their official platforms. We confirm that we are the rightful owner of the work and hold all necessary rights to grant this permission. We affirm that, reported work is original and not in consideration elsewhere.

Competing interests
The authors declare no competing interests.

Received: 7 July 2025 / Accepted: 10 December 2025
Published online: 03 February 2026

References

1. https://www.adnas.com/
2. Oya M, Shi Y, Yanagisawa M, Togawa N. A score-based classification method for identifying hardware-trojansat gate-level netlists, in 2015 Design, Automation & Test in Europe Conference & Exhibition (DATE), ser. DATE '15, 2015. p. 465–470.
3. Zhang X, Tehranipoor M. Case study: Detecting hardware trojans in third-party digital IP cores, in Hardware-Oriented Security and Trust (HOST), 2011 IEEE International Symposium on. 2011. p. 67–70.
4. Koushanfar F, Mirhoseini A. A unified framework for multimodal submodular integrated circuits trojan detection. IEEE Transactions on Information Forensics and Security. 2011;6(1):162–74.
5. Sergei Skorobogatov and Christopher Woods. Breakthrough silicon scanning discovers backdoor in military chip. In Proceedings of the 14th international conference on Cryptographic Hardware and Embedded Systems. 2012.
6. Checkoway S, Maskiewicz J, Garman C, Fried J, Cohney S, Green M, Heninger N, Weinmann R-P. Eric Rescorla, and Hovav Shacham. A Systematic Analysis of the Juniper Dual EC Incident. In Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. 2016.
7. Li W, Gascon A, Subramanyan P, Tan WY, Tiwari A, Malik S, Shankar N, Seshia S. Wordrev: Finding word-level structures in a sea of bit-level gates, in Hardware-Oriented Security and Trust (HOST), 2013 IEEE International Symposium on. 2013. p. 67–74.
8. Swar A, Schult DA. Exploring network structure, dynamics, and function using NetworkX. Mathematical modeling and theoretical division, In: Varoquaux, G., and Vaught, T, editors. 7th Python in Science Conference. Pasadena, CA: 2008. p. 11–15.
9. Subramanyan P, et al. Reverse engineering digital circuits using structural and functional analyses. IEEE Transactions on Emerging Topics in Computing. 2014;2(1):63–80.
10. Fyrbiak M et al. Hardware Reverse Engineering: Overview and Open Challenges, in IVSW, 2017.
11. Huang Z, Wang Q, Chen Y, Jiang X. A survey on machine learning against hardware trojan attacks: recent advances and challenges. IEEE Access. 2020;8:10796–826.
12. Shi Y, et al. A highly efficient method for extracting FSMs from flattened Gate-Level Netlist. In ISCAS; 2010. p. 2610–3.

13. Seum A, Chowdhury MR, Hossain FS. An efficient machine learning approach for hardware Trojan detection. In: ICCIT; 2022. p. 182–7.
14. Hossain FS, Seum A, Chowdhury MR, Ahmed F. ZDD: A zero delay deviation variability-aware golden-free hardware trojan detection using PUF. IEEE Transactions on Circuits and Systems I, Regular Papers. 2025;72(8):4153–66. https://doi.org/10.1109/TCSI.2025.3559805
15. Sharma M, Mishra A, Sapatnekar SS, Li et al. Reverse Engineering Circuits Using Behavioral Pattern Mining, in IEEE HOST, 2012. p. 83–88.
16. Subramanyan P, et al. Reverse engineering digital circuits using functional analysis. in DATE; 2013. p. 1277–80.
17. Gascon´ A, Subramanyan P, Dutertre B, Tiwari A, Jovanovic D, Malik S. Template-based circuit Understanding. FormalMethodsinComputer-Aided Design. Lausanne, Switzerland, 2014:83–90.
18. Chattopadhyay Anindita, Bisariya Siddharth, Sutrakar VK. A deep learning approach for hardware Trojan detection in netlist of Integrated circuits with graph neural network. In 2nd NIELIT's International Conference on Communication, Electronics and Digital Technologies. 2024.
19. Wang J, Zhai G, Gao H, Xu L, Li X, Li Z, et al. A hardware trojan detection and diagnosis method for gate-level netlists based on machine learning and graph theory. Electronics. 2023;13(1):59.
20. Tempo AJ, Jeong Y-J. FP-GNN: a graph neural network for hardware trojan detection in gate-level netlist. IEICE Trans Inf Syst. 2025;E108D3:295–8.
21. Takamaeda-Yamazaki S. Pyverilog: A python-based hardware design processing toolkit for veriloghdl. In International Symposium on Applied Reconfigurable Computing. 2015.
22. Schult DA. Exploring network structure, dynamics, and function using NetworkX. mathematical modeling and theoretical division, In: Varoquaux, G., and Vaught, T, editors, 7th Python in Science Conference. Pasadena, CA: 2008. p. 11–15.
23. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Chintala S. PyTorch: an imperative style, high-performance deep learning library. Adv Neural Inf Process Syst. 2019;32(1):8024–35.
24. Salmani H, Tehranipoor M, Trust-Hub. Available: https://trust-hub.org/home, Accessed: 16 August 2025.
25. Wolf C, Glaser J, Kepler J. Yosys-A Free Verilog Synthesis Suite. 2013.
26. Yasaei R, Yu S-Y, Al Faruque MA, GNN4TJ. Graph Neural Networks for Hardware Trojan Detection at Register Transfer Level, 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE), Grenoble, France: 2021. p. 1504–1509.
27. Anindita Chattopadhyay A, Patel S, Bisariya. Sowmya Sunkara and Vijay Kumar Sutrakar, Deep Learning based Graph Neural Network Technique for Hardware Trojan Detection at Register Transfer Level, 4th IEEE VLSI SATA 2024 International conference held at Bangalore, May 16–17, 2024.
28. Amid A, Biancolin D, Gonzalez A, Grubb D, Karandikar S, Liew H, Magyar A, Mao H, Ou A, Pemberton N, Rigge P, Schmidt C, Wright J, Zhao J, Shao YS, Asanović K, Nikolić B. Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs, IEEE Micro. 40(4):10–21.
29. Mishchenko A, Chatterjee S, Brayton R. Improvements to technology mapping for LUT-based FPGAs. In: Proceedings of the International Workshop on Logic and Synthesis (IWLS). Berkeley: UC Berkeley. 2007.
30. Charles Papon. 2018. VexRiscv: A Versatile RISC-V Softcore CPU [Online]. Available at: https://github.com/SpinalHDL/VexRiscv
31. Data set is available at. https://github.com/aninchatto/Fast-and-Accurate-Identification-of-Hardware-Trojan-Locations-in-Gate-Level-Netlist-using-ML
32. Zhang S, Zhou S, Lü L, Jia C, Wang J. Hardware trojan detection with feature fusion, in Proc. IEEE 24th Int Conf Softw Qual Rel Secur Companion (QRS-C), Jul. 2024. p. 1324–1325.
33. Cai TT, Ma R. Theoretical foundations of t-SNE for visualizing high-dimensional clustered data. J Mach Learn Res. 2022;23(301):1–54.
34. McInnes L, Healy J, Melville J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction, arXiv preprint arXiv:1802.03426, 2018.
35. Antón ALD, Müller J, Deutschmann L, Fadiheh MR, Stoffel D, Kunz W. A golden-free formal method for trojan detection in noninterfering accelerators, in Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE), Mar. 2024. p. 1–6.
36. Li Y et al. Aug., Pinpointing hardware trojans through semantic feature extraction and natural language processing, in Proc. IEEE Int Test Conf Asia (ITC-Asia), 2024. p. 1–6.
37. Sutikno S, Putra SD, Wijitrisnanto F, Aminanto ME. Detecting unknown hardware trojans in register transfer level leveraging verilog conditional branching features. IEEE Access. 2023;11:46073–83.
38. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G. The graph neural network model. IEEE Trans Neural Networks. 2009;20(1):61–80.
39. Yasaei R, Yu S-Y. and M. A. Al Faruque, Hardware Trojan Detection Using Graph Neural Network, IEEE Transactions on Computer-Aided Design of Integrated Circuits and System .2025;44(1):25-38 https://doi.org/10.1109/TCAD.2022.3178355}
40. Kento Hasegawa Y, SeiraHidano K, Fukushima K, Hashimoto. March, NozomuTogawa, Node-Wise hardware Trojan detection based on graph learning. IEEE Trans Comput. 2025:74(3):749-761.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.